

Reporte de Avance 1: Adversarial Camouflage para Evasión de Detectores de Objetos en Imágenes Aéreas

Diederich Solís - Christian Echeverría

22 de mayo de 2026

1. Introducción

El presente documento detalla los avances iniciales correspondientes al desarrollo del proyecto de investigación titulado *Adversarial Camouflage para Evasión de Detectores de Objetos en Imágenes Aéreas: Diseño, Optimización y Evaluación bajo Condiciones Físicas Reales*. En esta primera etapa, el objetivo principal ha sido establecer una base técnica sólida, configurar el pipeline de datos, y validar el funcionamiento del modelo de detección sobre el cual se ejecutará el ataque adversarial.

2. Avances Técnicos Alcanzados

2.1. 1. Configuración de Infraestructura y Entorno

Debido a la alta demanda computacional requerida para la optimización adversarial y el gran volumen de datos del dataset DOTA v1.0, se ha completado la configuración del entorno de trabajo:

- **Entorno en la Nube:** Se configuró un entorno en Google Colab con soporte para aceleración por hardware (GPU CUDA), validado mediante scripts de comprobación de entorno (`check_runtime.py`).
- **Gestión de Datos:** Se diseñó un pipeline híbrido. El código fuente y el versionamiento se manejan a través de GitHub, mientras que los datos pesados (imágenes crudas de DOTA y anotaciones) se gestionan mediante una integración con Google Drive para evitar la saturación del repositorio.
- **Estructura del Proyecto:** Se implementó la arquitectura base del paquete en Python (`src/camouflage`) siguiendo buenas prácticas de ingeniería de software para investigación, incluyendo los esqueletos para los scripts de entrenamiento y evaluación digital/física.

2.2. 2. Implementación del Modelo Baseline (YOLOv8 OBB)

Antes de generar la textura adversarial, es imperativo contar con un modelo de detección funcional que sirva como *baseline* (línea base).

- Se logró instanciar y ejecutar con éxito el modelo **YOLOv8n-obb** (Oriented Bounding Boxes), el cual está diseñado específicamente para la detección de objetos con rotación en vistas cenitales (imágenes aéreas y satelitales).
- Se implementaron scripts (`run_yolo_baseline.py`) para ejecutar inferencias sobre imágenes limpias y obtener las métricas base antes del ataque.

2.3. 3. Prueba de Concepto Visual (Overlay Adversarial)

Como paso preliminar a la optimización matemática de la textura mediante gradientes, se desarrolló una prueba de concepto (`01_overlay_demo.ipynb`) que demuestra la manipulación programática de las imágenes:

- **Extracción de Máscaras:** El script ejecuta el modelo YOLOv8 OBB, extrae las coordenadas de los polígonos detectados (vehículos) y genera máscaras precisas sobre la ubicación del objeto.
- **Inyección de Ruido:** Se inyecta una textura sintética (ruido aleatorio) de forma delimitada utilizando las máscaras extraídas.
- **Re-evaluación:** Se somete la imagen intervenida nuevamente al modelo de detección para observar el comportamiento y la degradación de la confianza del detector ante una oclusión estructurada. Esta lógica servirá como el *canvas* donde se proyectará la textura optimizada matemáticamente.

3. Próximos Pasos

Habiendo consolidado la infraestructura y la manipulación de imágenes, las siguientes fases del proyecto se enfocarán en:

1. Implementar la función de pérdida adversarial (Adversarial Loss) orientada a minimizar la confianza de clasificación del modelo YOLOv8.
2. Integrar el módulo de transformaciones diferenciables usando *Kornia* para aplicar el método *Expectation Over Transformation (EoT)*, simulando variaciones de altitud y ángulos.
3. Iniciar el ciclo de optimización por gradientes para generar la primera versión convergente de la textura de camuflaje.